

BÁO CÁO BÀI TẬP LỚN

Hệ thống đấu giá trực tuyến - Nhóm 9

Môn: Lập trình nâng cao | Kiến trúc: JavaFX MVC, TCP Socket, DAO, MySQL + HikariCP

1. Giới thiệu mục tiêu và phạm vi

Đề tài xây dựng một hệ thống đấu giá trực tuyến nhiều người dùng theo mô hình client-server. Người dùng đăng ký, đăng nhập và tham gia theo ba vai trò Admin, Seller và Bidder. Seller đăng sản phẩm và gửi yêu cầu mở phiên; Admin duyệt tài khoản, phiên đấu giá và yêu cầu nạp tiền; Bidder nạp tiền, đặt giá thủ công hoặc tự động, theo dõi cập nhật realtime và thanh toán khi thắng.

Phạm vi triển khai tập trung vào quy trình đấu giá đầy đủ: quản lý tài khoản, quản lý sản phẩm, đặt giá, đấu giá tự động, chống sniping cuối phiên, tự động kết thúc phiên, xác định người thắng, thanh toán qua ví nội bộ, lưu trữ dữ liệu và kiểm thử nghiệp vụ. Hệ thống không tích hợp cổng thanh toán thật; số dư được mô phỏng trong database.

2. Kiến trúc tổng thể

Hệ thống được tổ chức theo kiến trúc phân tầng. Client JavaFX chịu trách nhiệm hiển thị và gửi request; server tiếp nhận payload JSON qua TCP socket, điều phối qua controller, xử lý nghiệp vụ tại manager và lưu dữ liệu qua DAO. Realtime update sử dụng Observer Pattern để đẩy sự kiện về các client đang kết nối.

Sơ đồ kiến trúc tổng thể

Client JavaFX FXML Views Client Controllers AppContext AuctionClient	TCP Socket JSON Payload ResponsePayload	Server Network AuctionServer ClientHandler Observer dispatch
	PayloadType router	Server Controllers AuthController AuctionController AdminController ProfileController
	Gọi nghiệp vụ	Business Layer AuthManager, ItemManager AuctionManager, AdminManager ProfileManager, AuctionScheduler
	Transaction + Lock	DAO Layer UserDAO, ItemDAO, AuctionDAO BidDAO, AutoBidDAO DepositRequestDAO, ProfileDAO
	SQL / File I/O	Data Storage MySQL Database data/uploads/items

Client JavaFX: Login, Register, Bidder, Seller, Admin; FXML + Controller theo MVC
AuctionClient: gửi Payload JSON qua TCP socket, nhận ResponsePayload và realtime update
AuctionServer + ClientHandler: 1 thread/client, Observer đẩy cập nhật tới client
Server Controller: AuthController, AuctionController, AdminController, ProfileController điều phối request
Manager/BLL: AuthManager, ItemManager, AuctionManager, AdminManager, ProfileManager; lock theo item + transaction

DAO: UserDao, ItemDao, AuctionDao, BidDao, AutoBidDao, DepositRequestDao, ProfileDao

Database MySQL + thư mục data/uploads/items lưu ảnh sản phẩm

Mô tả theo kiến trúc

- Tầng giao diện: JavaFX + FXML theo MVC, gồm Login, Register, Bidder, Seller, Admin và màn hình đấu giá.
- Tầng giao tiếp: AuctionClient và AuctionServer trao đổi Payload/ResponsePayload JSON qua TCP socket.
- Tầng điều phối: AuthController, AuctionController, AdminController, ProfileController kiểm tra request và gọi manager phù hợp.
- Tầng nghiệp vụ: AuctionManager, AuthManager, ItemManager, AdminManager, ProfileManager xử lý luật đấu giá, ví, duyệt, hồ sơ và lifecycle phiên.
- Tầng dữ liệu: DAO tách truy vấn SQL, HikariCP quản lý connection pool, database lưu người dùng, sản phẩm, phiên và lịch sử bid.

Các quyết định thiết kế chính

- Singleton cho các manager và Database để có điểm truy cập nghiệp vụ thống nhất.
- Factory Method cho ItemFactory nhằm tạo sản phẩm theo loại Electronics, Art, Vehicle, Book, Fashion, Home, Collectible.
- Observer cho realtime update giữa server và client.
- ReentrantLock theo item kết hợp transaction DB để đảm bảo an toàn khi nhiều bidder đặt giá cùng lúc.

3. Chức năng đạt được theo barem

Bảng dưới đây tóm tắt chức năng, hướng giải quyết và lý do lựa chọn. Các chức năng nâng cao đã triển khai gồm auto-bidding và anti-sniping; bid history hiện hiển thị dạng lịch sử văn bản.

Chức năng	Hướng giải quyết	Lý do lựa chọn
Quản lý người dùng	User kế thừa thành Admin/Seller/Bidder; AuthManager và UserDao xử lý đăng ký, đăng nhập, duyệt seller.	Tách vai trò rõ ràng, đúng OOP và dễ mở rộng.
Quản lý sản phẩm	Seller CRUD sản phẩm; ItemFactory + ItemCreator tạo nhiều category; ảnh lưu file, metadata lưu DB.	Giảm phụ thuộc UI vào logic tạo đối tượng, phù hợp Factory Method.
Mở và duyệt phiên	Seller gửi yêu cầu kèm start/end time; Admin duyệt; server chuyển OPEN -> RUNNING.	Đảm bảo seller không tự mở phiên nếu chưa được kiểm soát.
Đặt giá hợp lệ	AuctionManager kiểm tra RUNNING, giá cao hơn hiện tại, không tự đấu sản phẩm của mình, đủ số dư.	Luật đấu giá tập trung ở server để tránh client tự ý thay đổi.
Realtime update	AuctionServer cài Observer; ClientHandler nhận update và gửi ResponsePayload về client đang kết nối.	Client thấy giá, trạng thái và số dư mới mà không cần refresh.
Kết thúc phiên	AuctionScheduler quét phiên hết hạn, gọi finishAuction; xác định winner từ highestBidderId, xử lý unsold/canceled.	Tự động hóa vòng đời phiên, kể cả khi không có thao tác thủ công.
Concurrency	ReentrantLock theo item + transaction DB khi đặt giá, auto-bid, kết thúc hoặc thanh toán.	Chống race condition/lost update khi nhiều bidder đặt giá đồng thời.
Auto-bidding	Lưu maxBid và increment; ưu tiên maxBid cao hơn, nếu bằng nhau ưu tiên tạo trước; server tự sinh bid.	Mô phỏng sàn đấu giá thực, vẫn giữ logic ở server để nhất quán.
Anti-sniping	Nếu bid nằm trong 60 giây cuối, hệ thống kéo end_time thêm 60 giây trong cùng transaction.	Tránh thắng bằng cách đặt giá sát giờ đóng; client nhận thời hạn mới qua realtime.

Chức năng	Hướng giải quyết	Lý do lựa chọn
Ví và thanh toán	Bidder nạp tiền qua request; Admin duyệt; winner thanh toán thì trừ bidder và cộng seller.	Mô phỏng luồng thanh toán khép kín mà không cần cổng thanh toán thật.
Hồ sơ người dùng	ProfileController/Manager/DAO tổng hợp thông kê bidder, seller, lịch sử bid và kết quả bán.	Tách phần truy vấn thống kê khỏi luồng đầu giá chính, UI đọc qua payload riêng.
Lưu trữ dữ liệu	DAO + HikariCP lưu users, items, auctions, bids, auto_bids, deposit_requests; ảnh tách ra thư mục uploads.	Dữ liệu bền vững, tách truy cập DB khỏi nghiệp vụ.
Kiểm thử và CI/CD	JUnit 5 test model/manager/DAO/concurrency; Maven, JaCoCo, Checkstyle, GitHub Actions.	Tự động phát hiện lỗi logic và chứng minh chất lượng theo yêu cầu đề.

4. Kiểm thử, build và mức độ hoàn thiện

Dự án dùng Maven để build, JUnit 5 để kiểm thử, JaCoCo để đo coverage và Checkstyle/Qodana để hỗ trợ kiểm soát chất lượng mã. Các test tập trung vào model, manager, DAO, auto-bid và concurrency. Lệnh kiểm tra tổng thể mvn verify đã chạy thành công trên môi trường hiện tại.

Mức độ hoàn thiện: các chức năng bắt buộc của đề đã được triển khai đầy đủ ở mức demo được. Hai chức năng nâng cao đã có là auto-bidding và anti-sniping. Nếu tiếp tục mở rộng, phần còn lại nên bổ sung biểu đồ bid history realtime bằng JavaFX LineChart.